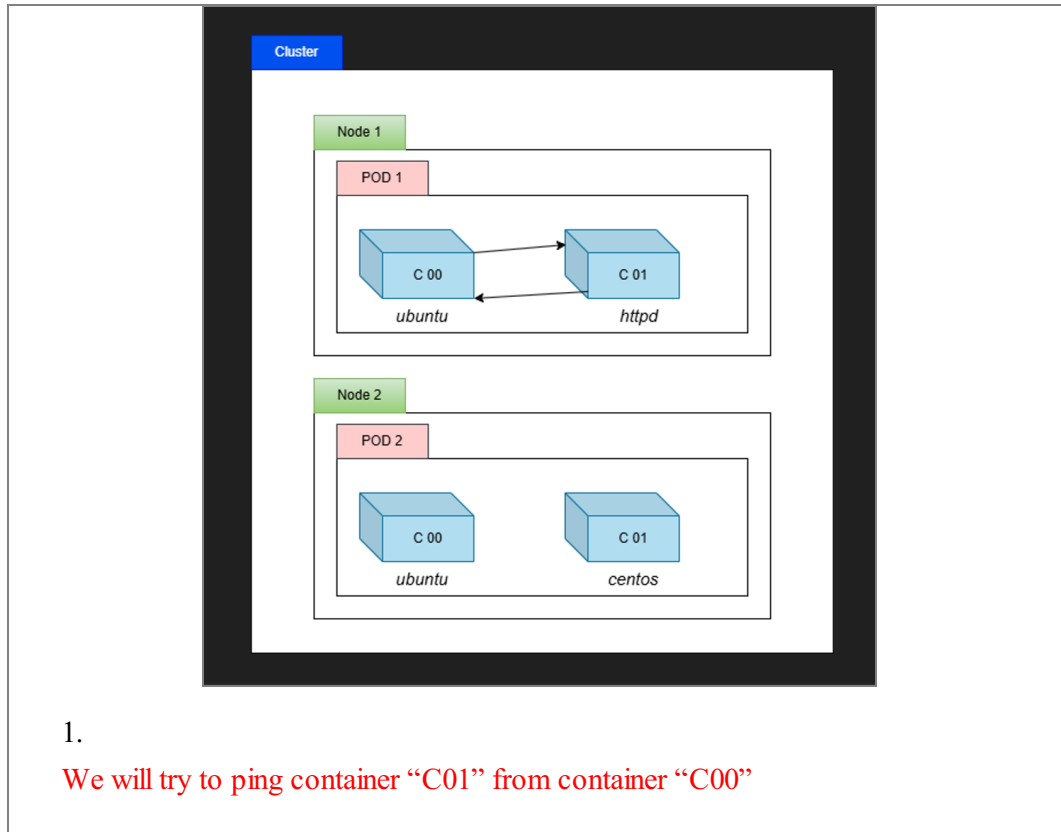

NETWORKING IN KUBERNETES

Rohit Surwade

Testing connection/communication between two containers in the same POD.



Manifest to create two containers in the same POD.

```
root@ip-172-31-36-215: /home/ubuntu
apiVersion: v1
kind: Pod
metadata:
  name: testpod
spec:
  containers:
  - name: c00
    image: ubuntu
    command: ["/bin/bash", "-c", "while true; do echo Hello-Rohit; sleep 5; done"]
  - name: c01
    image: httpd
    ports:
    - containerPort: 80
```

2.

KUBERNETES NETWORKING

(This kind of manifest is useful for testing basic Kubernetes features, such as running multiple containers inside a Pod and setting up simple services (like an HTTP server). We have defined a Kubernetes Pod named testpod.

Inside this Pod, there are two containers:

c00 (Ubuntu-based container) that runs a shell script printing "Hello-Bhupinder" every 5 seconds in an infinite loop. This serves as a simple test container.

c01 (Apache HTTP Server container) that exposes port 80 and runs a web server. This container will serve web content on port 80.)

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod
spec:
  containers:
  - name: c00
    image: ubuntu
    command: ["/bin/bash", "-c", "while true; do echo Hello-Rohit; sleep 5; done"]
  - name: c01
    image: httpd
    ports:
    - containerPort: 80
```

Applying the manifest and entering the C00 container to ping C01 container

```
root@ip-172-31-36-215:/home/ubuntu# minikube start --force
* minikube v1.34.0 on Ubuntu 24.04 (xen/amd64)
! minikube skips various validations when --force is supplied; this may lead to unexpected behavior
* Using the docker driver based on existing profile
* The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
* If you are running minikube within a VM, consider using --driver=none:
* https://minikube.sigs.k8s.io/docs/reference/drivers/none/
* Tip: To remove this root owned cluster, run: sudo minikube delete
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.45 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.31.0 on Docker 27.2.0 ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
root@ip-172-31-36-215:/home/ubuntu# A pod1.yml
root@ip-172-31-36-215:/home/ubuntu# B kubectl apply -f pod1.yml
pod/testpod created
root@ip-172-31-36-215:/home/ubuntu# C kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
testpod   2/2     Running   0           7s
root@ip-172-31-36-215:/home/ubuntu# kubectl exec testpod -it -c c00 -- /bin/bash
error: unknown flag: -- /bin/bash
See 'kubectl exec --help' for usage
root@ip-172-31-36-215:/home/ubuntu# kubectl exec testpod -it -c c00 -- /bin/bash
root@testpod:/#
```

3.

commands:

1. kubectl apply -f yaml-file-name.yml
2. kubectl get pods
3. kubectl exec pod-name -it -c container-name -- /bin/bash

Installing curl on container C00

```
root@testpod:/# A apt update && apt install curl
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [607 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [15.3 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [623 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [725 kB]
26% [6 Packages 921 kB/1808 kB 51%]
```

4.

apt update && apt install curl

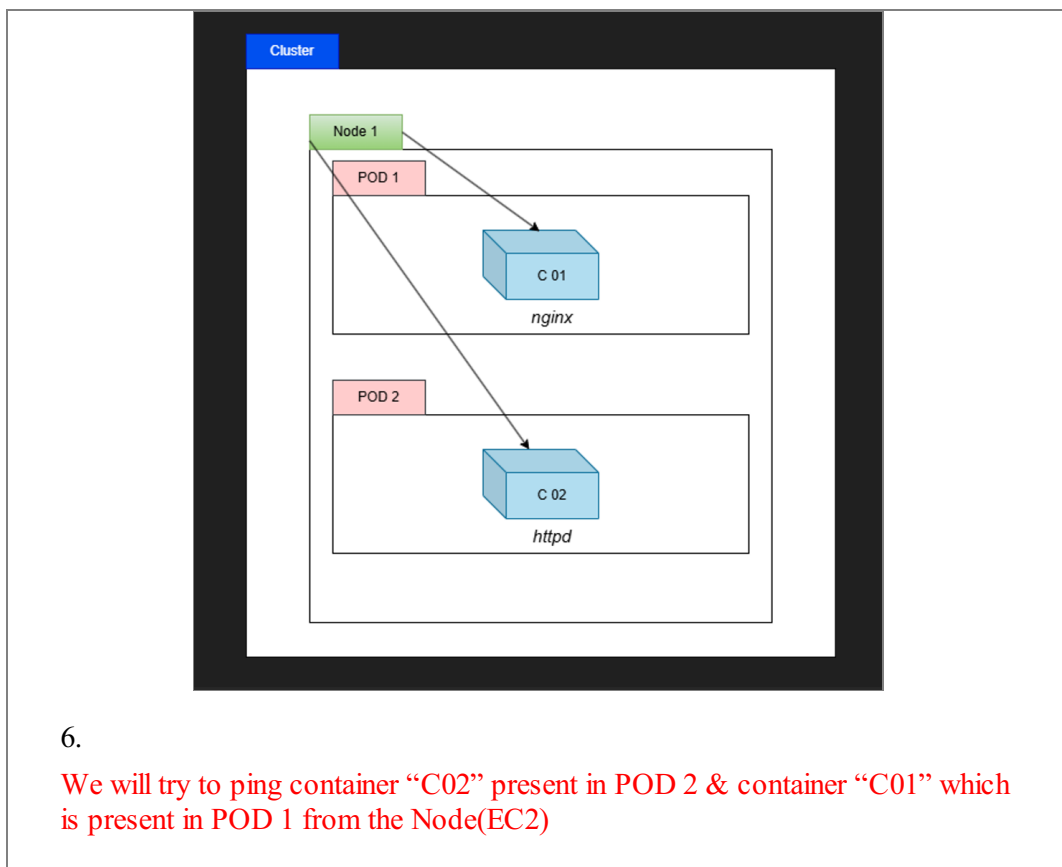
How to ping container C01

```
root@testpod:/# curl localhost:80
<html><body><h1>It works!</h1></body></html>
root@testpod:/#
```

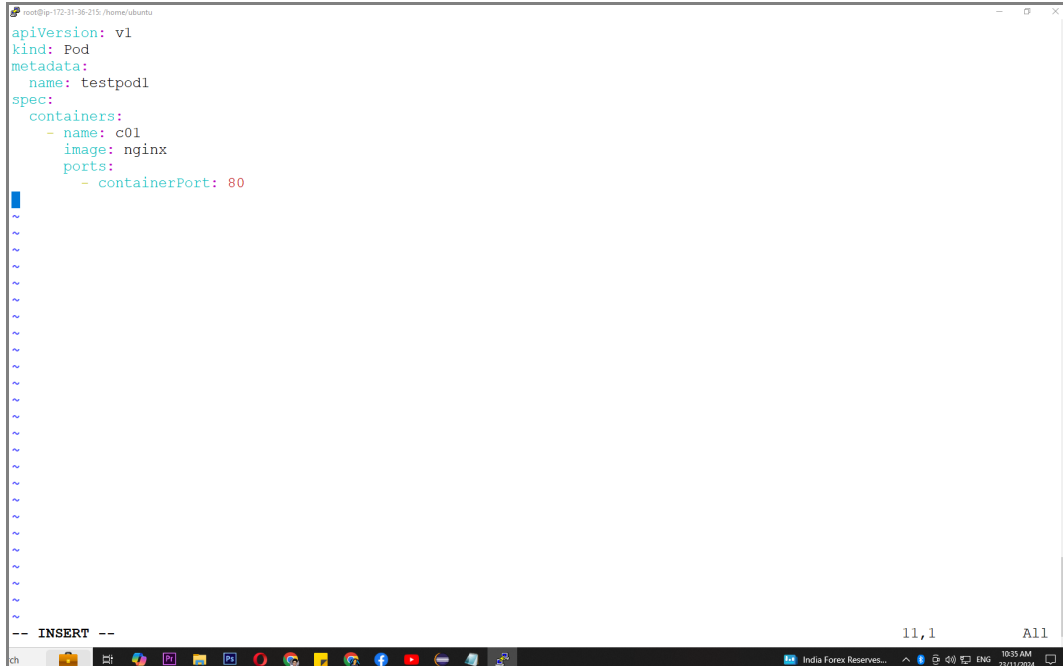
5.

```
curl localhost:80
```

Trying to establish communication between two different PODS within the same Node



Manifest to create a POD with a container "C01" with "NGINIX" image running on it.



```
root@ip-172-31-36-215: /home/ubuntu
apiVersion: v1
kind: Pod
metadata:
  name: testpod1
spec:
  containers:
    - name: c01
      image: nginx
      ports:
        - containerPort: 80
```

7.

KUBERNETES NETWORKING

(If you deploy this Pod in a Kubernetes cluster, Nginx will start running and wait for incoming HTTP requests on port 80.

You can access this service by creating a Kubernetes Service that exposes port 80, or you can use port forwarding to access it locally for testing.

containerPort: 80: This tells Kubernetes that the container will expose port 80. Port 80 is the default port for HTTP traffic, and since Nginx is a web server, it will listen for HTTP requests on this port.)

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod1
spec:
  containers:
    - name: c01
      image: nginx
      ports:
        - containerPort: 80
```

Manifest to create a POD with a container "C02" with "HTTPD" image running on it.

```
root@ip-172-31-36-215: /home/ubuntu
apiVersion: v1
kind: Pod
metadata:
  name: testpod2
spec:
  containers:
  - name: c02
    image: httpd
    ports:
    - containerPort: 80
```

8.

KUBERNETES NETWORKING

(This manifest defines a Kubernetes Pod named testpod2. The Pod contains a single container (c02) running an Apache HTTP Server (httpd image). The Apache web server inside the container listens on port 80, which is the default port for serving web content.)

```
apiVersion: v1
kind: Pod
metadata:
  name: testpod2
spec:
  containers:
  - name: c02
    image: httpd
    ports:
    - containerPort: 80
```

Successfully pinged container “C01” present in POD 1 from the Node

```

root@ip-172-31-36-215:/home/ubuntu# kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
testpod1   1/1     Running   1 (81m)    119m  10.244.0.51   minikube  <none>            <none>
testpod2   1/1     Running   1 (81m)    118m  10.244.0.52   minikube  <none>            <none>
root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.244.0.51:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@ip-172-31-36-215:/home/ubuntu#

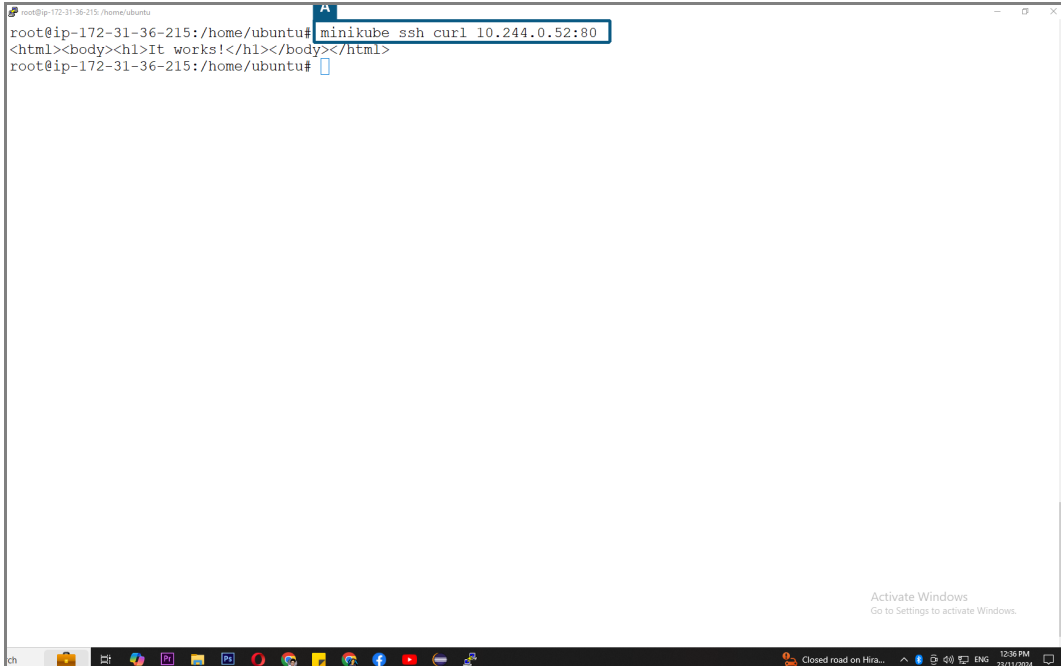
```

9.

commands:

1. kubectl get pods -o wide(to get ip adress related info).
2. minikube ssh curl IP:port no.

Successfully pinged container “C02” present in POD 2 from the Node



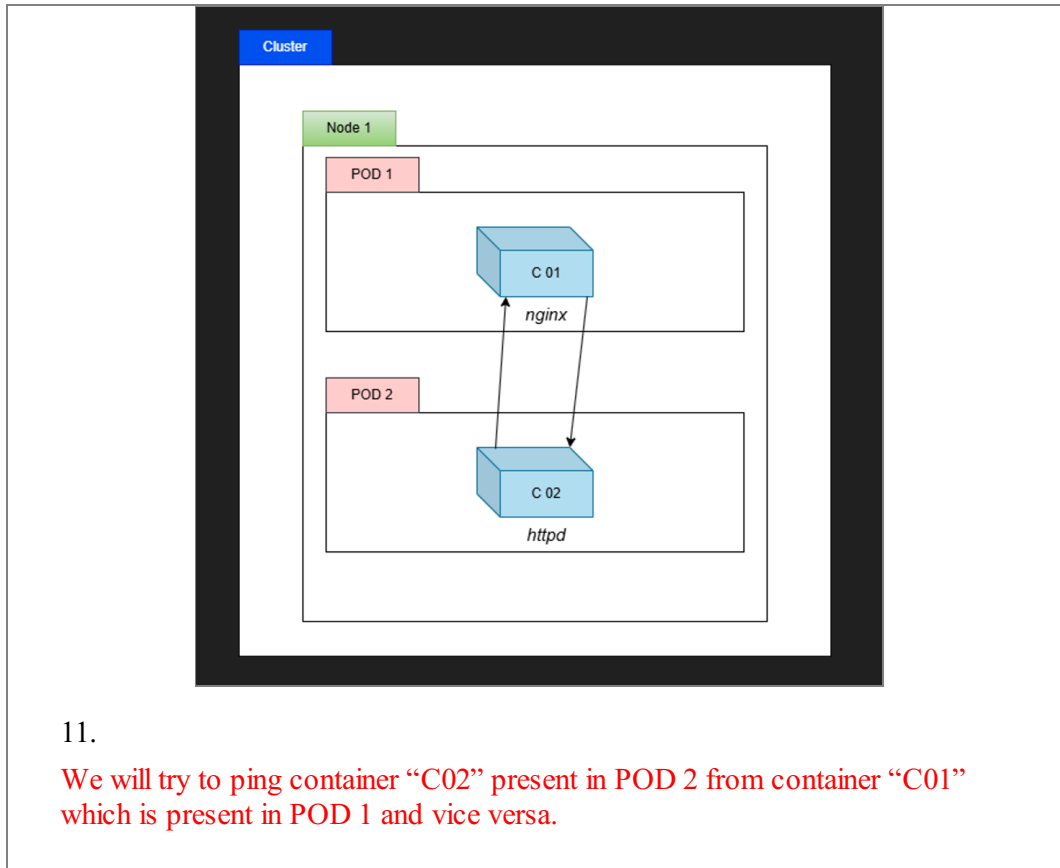
```
root@ip-172-31-36-215:/home/ubuntu# minikube ssh curl 10.244.0.52:80
<html><body><h1>It works!</h1></body></html>
root@ip-172-31-36-215:/home/ubuntu#
```

10.

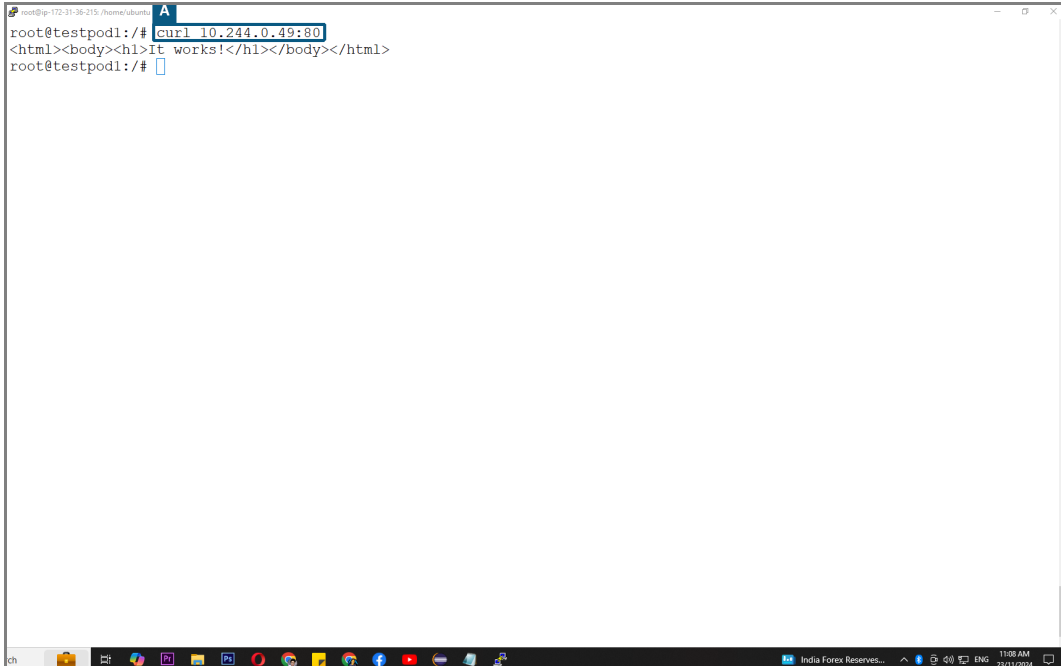
commands:

1. `kubectl get pods -o wide`(to get ip adress related info).
2. `minikube ssh curl IP:port no.`

Trying to Establish communication between different containers in different PODS within the same Node



Successfully pinged container “C02” present in POD 2 from container “C01” which is present in POD 1



```
root@ip-172-31-36-215: /home/ubuntu
root@testpod1:/# curl 10.244.0.49:80
<html><body><h1>It works!</h1></body></html>
root@testpod1:/#
```

12.

curl ip:port

NOTE:
we will have to install curl on the container first

Successfully pinged container “C01” present in POD 1 from container “C02” which is present in POD 2

```
root@ip-172-31-36-215:/home/ubuntu# curl
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 315 kB of archives.
After this operation, 500 kB of additional disk space will be used.
Get:1 http://deb.debian.org/debian bookworm/main amd64 curl amd64 7.88.1-10+deb12u8 [315 kB]
Fetched 315 kB in 0s (18.5 MB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package curl.
(Reading database ... 6791 files and directories currently installed.)
Preparing to unpack .../curl_7.88.1-10+deb12u8_amd64.deb ...
Unpacking curl (7.88.1-10+deb12u8) ...
Setting up curl (7.88.1-10+deb12u8) ...
root@testpod2:/usr/local/apache2# curl 10.244.0.48:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@testpod2:/usr/local/apache2#
```

13.

curl ip:port

NOTE:
we will have to install curl on the container first